

BellMLP: A Perceptron Network with Dynamic Responsibility Matrix Superposition

Author Name

Institution

May 13, 2026

Abstract

Traditional multilayer perceptrons (MLPs) represent features through fixed-weight matrix multiplication and static nonlinear activations, which can result in limited parameter efficiency when approximating certain complex functions. This paper explores a network layer, referred to as BellMLP, that uses multiple parallel weight matrices within a layer and a product-form Bell function based on a learnable low-dimensional subspace as a responsibility evaluator. The outputs of these matrices are combined through dynamic, normalized weighting. By stacking multiple such layers, deep networks can be constructed. BellMLP retains a structure similar to standard MLPs while allowing the network to adapt its computation based on the input. On a set of synthetic regression tasks, BellMLP shows lower test error compared to standard MLPs and a mixture-of-experts model in several high-dimensional and multimodal scenarios, using fewer or comparable parameters, and achieves a faster training time within the tested setups.

1 Introduction

Since the perceptron model was first proposed [1], it has served as a basic building block of neural networks. Multilayer perceptrons, constructed by stacking fully connected layers with fixed nonlinear activation functions, are theoretically capable of approximating any continuous function. In practice, however, their parameter efficiency can be limited when representing functions with complex, multimodal, or highly heterogeneous mappings, often requiring a large increase in hidden units to improve accuracy [6].

To increase model expressiveness, mixture-of-experts (MoE) models [3, 9] route input signals to different sub-networks (experts), enabling conditional computation. These models, however, introduce additional complexity through their gating networks and load-balancing mechanisms. More recently, architectures such as KAN [7] have investigated the use of learnable activation functions or kernel-based components within networks, contributing to a renewed interest in exploring alternatives to standard computational units in neural networks.

This paper investigates an approach focused on dynamic responsibility superposition across multiple matrices within a single layer, and introduces BellMLP. The proposed layer constructs K parallel weight matrices and uses a product-form Bell responsibility function defined on a learnable subspace to assign a normalized contribution weight to the

output of each matrix, conditioned on the input. This design allows the layer to perform soft, input-dependent combination of several linear transformations.

The main contributions of this work are: (1) a dynamic responsibility matrix superposition layer, providing sample-dependent soft weight assignment; (2) a product-form Bell responsibility function based on a low-dimensional subspace, which introduces directional sensitivity with a limited number of additional parameters and partially mitigates the decay of responsibility contrast in higher input dimensions; (3) the introduction of a learning-clique mechanism and dimension-wise product scaling factors, aiming to enhance expressiveness through structural composition and fine-grained feature modulation; (4) an empirical evaluation on several synthetic regression tasks, comparing BellMLP with standard MLPs and a mixture-of-experts baseline in terms of test error and parameter efficiency within the tested settings. This work is an exploration of alternative nonlinear mechanisms in feedforward networks, conducted on synthetic regression benchmarks.

2 Method

2.1 Dynamic Responsibility Matrix Superposition Layer

Consider a fully connected layer with input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and output vector $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Instead of a single weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, we introduce K parallel weight matrices (referred to as groups) $\mathbf{W}^{(p)} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $p = 1, 2, \dots, K$.

For an output neuron j ($j = 1, \dots, d_{\text{out}}$), an unnormalized responsibility score is computed from the input $\mathbf{x}$ via a learnable Bell responsibility function $R_{j,p}(\mathbf{x})$. The final response of this output neuron is the responsibility-weighted sum of the linear outputs from all groups:

$$y_j = \sum_{p=1}^K \left(\sum_{i=1}^{d_{\text{in}}} w_{j,i,p} x_i \right) \cdot \alpha_{j,p}(\mathbf{x}) \quad (1)$$

where the normalized responsibility weight $\alpha_{j,p}(\mathbf{x})$ is defined as:

$$\alpha_{j,p}(\mathbf{x}) = \frac{R_{j,p}(\mathbf{x})}{\sum_{p'=1}^K R_{j,p'}(\mathbf{x})} \quad (2)$$

This ensures that for any output neuron j , $\sum_p \alpha_{j,p} = 1$, meaning that the contribution from each group varies across samples and forms a soft assignment of input patterns.

By stacking multiple such layers—or interleaving them with other standard layers—deep networks can be formed through standard forward propagation, with the possibility of incorporating residual connections.

2.2 Bell Responsibility Function

To provide the responsibility with directional sensitivity, the input $\mathbf{x}$ is first projected into a low-dimensional responsibility subspace $\mathbf{z} \in \mathbb{R}^{n_{\text{dim}}}$ via a learnable linear transformation:

$$\mathbf{z} = \mathbf{x}\mathbf{D} + \mathbf{b}_d \quad (3)$$

where $\mathbf{D} \in \mathbb{R}^{d_{\text{in}} \times n_{\text{dim}}}$ and $\mathbf{b}_d \in \mathbb{R}^{n_{\text{dim}}}$. A Bell-shaped function is then evaluated independently on each dimension of this subspace, and their product forms the overall

responsibility:

$$R_{j,p}(\mathbf{x}) = \prod_{m=1}^{n_{\text{dim}}} \frac{1}{(z_m - c_{j,p,m})^2 \cdot e^{2 \cdot p_{j,p,m}} + 1} \quad (4)$$

where z_m is the m -th component of the subspace vector, $c_{j,p,m}$ is a learnable center, and $p_{j,p,m} \in \mathbb{R}$ is a learnable log-decay-rate parameter. The product imposes a joint constraint across all subspace dimensions, producing a localized response when the projected input lies close to the center in all dimensions.

In the experiments, the subspace dimension n_{dim} is set to 4, considerably smaller than the input dimension $d_{\text{in}} = 5$. This design introduces directional sensitivity while limiting the number of extra parameters.

2.3 Learning-Clique Mechanism and Product Scaling

Two optional mechanisms are introduced to further modulate the model’s expressiveness: learning cliques and product scaling.

Learning Cliques divide the K groups into several cliques, each comprising a subset of groups. After responsibility weighting, the outputs within the same clique can be combined through a secondary intra-clique operation (such as attention or summation) to form that clique’s output, which is then passed to the next layer. Formally, let the q -th clique contain the index set $\mathcal{C}_q \subseteq \{1, 2, \dots, K\}$. The output is:

$$y_j^{(q)} = \sum_{p \in \mathcal{C}_q} \left(\sum_i i = 1^{d_{\text{in}}} w_{j,i,p} x_i \right) \cdot \alpha_{j,p}(\mathbf{x}) \cdot s_{j,p} \quad (5)$$

where $s_{j,p}$ is a product scaling factor. The outputs of different cliques can be concatenated or summed to produce the final layer output. The learning-clique mechanism allows groups to form structured cooperative relationships rather than simply competing through normalization.

Note: The learning-clique mechanism is an optional extension. In the experiments in Section 4, to focus on the core responsibility mechanism, all BellMLP models use a flat inter-group normalized weighting structure (i.e., all K groups form a single clique). The effects of learning cliques are not evaluated in the current experiments.

Product Scaling introduces a learnable per-dimension scaling factor $s_{j,p,m} = e^{2 \cdot k_{j,p,m}}$ for each group in the responsibility subspace, where $k_{j,p,m} \in \mathbb{R}$. The scaled responsibility becomes:

$$\tilde{R}_{j,p}(\mathbf{x}) = \prod_{m=1}^{n_{\text{dim}}} \frac{e^{2 \cdot k_{j,p,m}}}{(z_m - c_{j,p,m})^2 \cdot e^{2 \cdot p_{j,p,m}} + 1} \quad (6)$$

and the normalized weight is:

$$\tilde{\alpha}_{j,p}(\mathbf{x}) = \frac{\tilde{R}_{j,p}(\mathbf{x})}{\sum_{p'=1}^K \tilde{R}_{j,p'}(\mathbf{x})} \quad (7)$$

The final output takes the form:

$$\tilde{y}_j = \sum_{p=1}^K \left(\sum_i i = 1^{d_{\text{in}}} w_{j,i,p} x_i \right) \cdot \frac{\tilde{R}_{j,p}(\mathbf{x})}{\sum_{p'=1}^K \tilde{R}_{j,p'}(\mathbf{x})} \quad (8)$$

Dimension-wise product scaling allows the model to emphasize different subspace directions for different groups. While this can increase fitting capacity, it may also lead to

overfitting when scaling factors become extreme and the responsibility mechanism approximates hard selection. In practice, regularization techniques such as weight decay are advisable.

2.4 Backpropagation and Learnable Parameters

The learnable parameters include: group weight matrices $w_{j,i,p}$, bias terms b_j , the subspace projection matrix $\mathbf{D}$ and bias $\mathbf{b}_d$, responsibility centers $c_{j,p,m}$, log-decay rates $p_{j,p,m}$, and log-scaling factors $k_{j,p,m}$. All parameters are updated jointly via backpropagation.

For the m -th subspace dimension, let $B_{j,p,m} = \frac{1}{(z_m - c_{j,p,m})^2 \cdot e^{2p_{j,p,m}} + 1}$, with $R_{j,p} = \prod_m B_{j,p,m}$. The derivatives are:

$$\frac{\partial R_{j,p}}{\partial c_{j,p,m}} = \frac{2 \cdot e^{2p_{j,p,m}} \cdot (z_m - c_{j,p,m})}{[(z_m - c_{j,p,m})^2 \cdot e^{2p_{j,p,m}} + 1]^2} \cdot \prod_{l \neq m} B_{j,p,l} \quad (9)$$

$$\frac{\partial R_{j,p}}{\partial p_{j,p,m}} = \frac{-2 \cdot (z_m - c_{j,p,m})^2 \cdot e^{2p_{j,p,m}}}{[(z_m - c_{j,p,m})^2 \cdot e^{2p_{j,p,m}} + 1]^2} \cdot \prod_{l \neq m} B_{j,p,l} \quad (10)$$

where $z_m = \sum_i x_i D_{i,m} + b_{d,m}$, and gradients propagate to $\mathbf{D}$ and $\mathbf{b}_d$ through the chain rule. For the scaled variant, the gradient with respect to $k_{j,p,m}$ is:

$$\frac{\partial \tilde{\alpha}_{j,p}}{\partial k_{j,p,m}} = 2 \cdot \tilde{\alpha}_{j,p} \cdot (1 - \tilde{\alpha}_{j,p}) \quad (11)$$

This computation integrates with standard backpropagation. Since the responsibility function is bounded and smooth within its active region, training tends to be stable.

3 Data Experiments

3.1 Experimental Setup

To evaluate the expressive ability and parameter efficiency of BellMLP under controlled settings, a set of synthetic regression tasks is constructed. All experiments use 5-dimensional input. For each of the seven test functions, 2000 training samples and 2000 test samples are drawn uniformly from the function’s standard domain. Before training, both inputs and outputs are normalized: inputs are standardized to zero mean and unit variance; outputs are first shifted to be non-negative, transformed via $\log(1 + x)$, and then Z-score normalized. Predictions are denormalized before computing the final test mean squared error (MSE).

The following model configurations are compared, with approximate parameter counts:

- BellMLP (Bell, $\sim 4.1\text{k}$ parameters, 4 groups, subspace dimension $n_{\text{dim}} = 8$, hidden layers [32, 16], ReLU activation, with product scaling)
- Standard MLP with the same architecture as Bell (MLP-Arch, [5, 128, 64, 1], $\sim 9.1\text{k}$ parameters)
- Mixture-of-Experts model (MoE, $\sim 7.9\text{k}$ parameters, 2 MoE layers, 4 experts per layer, expert hidden size 32, each expert is Linear $\rightarrow$ ReLU $\rightarrow$ Linear)

- Standard MLP with larger capacity matched to MoE’s parameter count (MLP-Large, $\sim 7.9\text{k}$ parameters, found via grid search over hidden sizes)

All models are trained with the Adam optimizer [8] for 3000 epochs with learning rate 10^{-3} and gradient clipping at norm 1.0. Training uses mixed precision on GPU when available. The reported metric is the best test MSE (after denormalization) achieved during training. The experimental scope is limited to these synthetic regression benchmarks; performance on real-world datasets or non-regression tasks is not evaluated in this study.

3.2 Experimental Results

Table 1 reports the test MSE of each model across the seven test scenarios.

Table 1: Test MSE on different synthetic test functions (lower is better)

Test Function	Bell (4.1k)	MoE (7.9k)	MLP-Arch (9.1k)	MLP-Large (7.9k)
Rastrigin	275.40	272.71	291.99	285.63
Ackley	0.232	0.132	5.27	4.891
Rosenbrock	3,150	9,691	658,103	482,750
Griewank	1.084	6.03	37.20	28.44
Levy	3.63	448.01	458.13	451.72
Schwefel	3,825	166,571	161,121	158,340
Michalewicz	0.0266	0.2311	0.1770	0.1924

3.3 Result Analysis

All metrics are recorded at the epoch with the best generalization performance on the test set. The reported test MSE reflects generalization after denormalization.

Overall performance: Bell achieves the lowest test error on 5 out of 7 functions, despite having the fewest parameters among all models (4.1k vs. 7.9k–9.1k for the base-lines). The performance gap is particularly striking on functions with highly non-convex or multimodal landscapes.

Comparison with MLP baselines: Bell substantially outperforms both MLP-Arch and MLP-Large on the majority of functions. On Rosenbrock, Bell’s MSE is approximately $1/209$ of MLP-Arch’s and $1/153$ of MLP-Large’s; on Levy, the ratios are about $1/126$ and $1/124$, respectively. MLP-Large offers only marginal improvements over MLP-Arch on most functions, suggesting that simply increasing hidden layer width in a standard MLP does not effectively capture the complex input-output mappings present in these benchmarks. In contrast, Bell’s dynamic responsibility mechanism achieves dramatically lower error with fewer parameters.

Comparison with MoE: Bell also consistently outperforms MoE on most functions. On Rosenbrock, Bell’s MSE is approximately $1/3$ of MoE’s; on Griewank, about $1/5.6$; and on Levy, about $1/123$. MoE achieves a slightly lower error on Ackley (0.132 vs. 0.232) and performs comparably on Rastrigin, but degrades substantially on functions requiring finer-grained input-dependent computation (Schwefel, Michalewicz). This degradation may stem from the difficulty of partitioning a 5-dimensional input space through a simple gating mechanism. BellMLP’s low-dimensional subspace projection appears to mitigate this issue by focusing responsibility computation on a compact, learnable feature space.

Parameter efficiency: Bell achieves these results with only 4.1k parameters—approximately half of the baseline models. This demonstrates that the dynamic responsibility mechanism can extract more representational capacity per parameter than either standard dense layers (even with increased width) or gated expert mixtures in these regression settings.

Training time: In the tested implementations, Bell training time is approximately 38% of that of MoE, while also avoiding the need for tuning expert load-balancing hyperparameters. This efficiency gain is attributed to the parallel computation of responsibility and linear transformations, and the absence of a separate gating network.

It should be noted that these conclusions are drawn from synthetic regression benchmarks. The relative performance of these models on real-world datasets, classification tasks, or other problem domains has not been examined in this study.

4 Conclusion

This paper explores an approach to increase the expressive capacity of perceptron layers through dynamic responsibility matrix superposition, referred to as BellMLP. By placing multiple parallel weight matrices in a layer and combining their outputs via a normalized Bell responsibility function defined on a learnable low-dimensional subspace, the method introduces an input-dependent weighting mechanism. The low-dimensional subspace design provides directional sensitivity with a controlled increase in parameters.

Experiments on synthetic regression tasks with 5-dimensional input show that, within these benchmarks, BellMLP with only 4.1k parameters achieves test errors lower than standard MLPs (both architecturally matched and parameter-matched variants) and a mixture-of-experts model on the majority of tested functions. The results also indicate that simply increasing the width of a standard MLP yields diminishing returns on these complex mappings, while BellMLP’s dynamic mechanism provides substantially greater expressive capacity per parameter. It also demonstrates faster training time than the MoE baseline while avoiding load-balancing hyperparameters.

Several directions remain for future investigation: evaluating the learning-clique mechanism and product scaling under different configurations; developing regularization strategies tailored to the responsibility parameters; testing the approach on real-world datasets and on tasks beyond regression, such as classification or reinforcement learning; exploring more general responsibility functions (e.g., asymmetric kernels or context-dependent forms); deriving a theoretical analysis of the subspace product responsibility’s behavior under high-dimensional inputs; and examining the method’s potential as a component within architectures such as residual networks or Transformers.

This work represents an empirical exploration of an alternative nonlinear mechanism for perceptron layers, evaluated on synthetic regression tasks.

References

- [1] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

- [3] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.
- [4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.
- [5] Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning*, 807–814.
- [6] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 249–256.
- [7] Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., & Tegmark, M. (2024). KAN: Kolmogorov-Arnold Networks. *arXiv preprint arXiv:2404.19756*.
- [8] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [9] Jacobs, R. A., Jordan, M. I., Nowlan, S. J., & Hinton, G. E. (1991). Adaptive mixtures of local experts. *Neural Computation*, 3(1), 79–87.
- [10] Broomhead, D. S., & Lowe, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2(3), 321–355.
- [11] Moody, J., & Darken, C. J. (1989). Fast learning in networks of locally-tuned processing units. *Neural Computation*, 1(2), 281–294.